

COURSE SYSTEM MVC PROJEKAT

Korištenje AI alata u razvoju aplikacije

1. Uvod

U okviru ovog projekta razvijena je MVC web aplikacija koristeći .NET Framework i Entity Framework.

Aplikacija podržava dva tipa korisnika:

- **Admin** – upravlja sadržajem (kursevi, instruktori, lekcije, ispiti)
- **Korisnik** – pregled kurseva, upis, polaganje ispita, komentari

Tokom razvoja korišten je AI chat-bot kao pomoćni alat za:

- generisanje koda
- rješavanje grešaka
- objašnjenje SQL funkcija i trigera
- implementaciju dodatnih funkcionalnosti

U nastavku su prikazani ključni upiti (promptovi) i odgovori.

2. Ključni promptovi i odgovori

2.1 Autentikacija korisnika

Pitanje:

Kako implementirati login koristeći Session u ASP.NET MVC?

Odgovor:

Login se implementira tako što se nakon provjere korisnika u bazi podataka u Session čuvaju vrijednosti:

- UserID
- RoleID

Primjer:

```
Session["UserID"] = user.UserID;
```

```
Session["RoleID"] = user.RoleID;
```

Na osnovu ovih vrijednosti aplikacija prepoznaje da li je korisnik prijavljen i koja je njegova uloga.

2.2 Ograničavanje pristupa (Admin)

Pitanje:

Kako ograničiti pristup određenim akcijama samo adminu?

Odgovor:

Koristi se provjera Session vrijednosti:

```
private bool IsAdmin()
{
    return Session["RoleID"] != null && (int)Session["RoleID"] == 1;
}
```

U svakoj akciji:

```
if (!IsAdmin())
    return RedirectToAction("Index", "Home");
```

2.3 CRUD operacije

Pitanje:

Kako napraviti CRUD operacije koristeći Entity Framework?

Odgovor:

- Create → db.Add()
- Read → db.ToList()
- Update → db.Entry().State = Modified
- Delete → db.Remove()

Primjer brisanja:

```
db.Courses.Remove(course);  
db.SaveChanges();
```

2.4 Brza i napredna pretraga

Pitanje:

Kako implementirati search u MVC aplikaciji?

Odgovor:

Brza pretraga koristi samo naziv:

```
courses = courses.Where(c => c.Title.Contains(title));
```

Napredna pretraga uključuje više filtera:

- kategorija
 - cijena (od-do)
-

2.5 SQL funkcija (Course Completed)

Pitanje:

Kako funkcija fn_IsCourseCompleted radi?

Odgovor:

Funkcija provjerava da li korisnik ima položene sve ispite iz kursa:

```
IF NOT EXISTS (
```

```
    SELECT ...
```

```
    WHERE Score IS NULL OR Score < 50
```

```
)
```

```
SET @Result = 1
```

Ako nema nepoloženih ispita → kurs je završen.

2.6 SQL Trigger (Notifications)

Pitanje:

Kako napraviti trigger za notifikacije nakon rezultata?

Odgovor:

Trigger se izvršava nakon INSERT u Results:

```
INSERT INTO Notifications (Message, UserID)
```

```
SELECT
```

```
    CASE
```

```
        WHEN Score >= 50 THEN 'You passed'
```

```
        ELSE 'You failed'
```

```
    END,
```

```
    UserID
```

```
FROM inserted
```

2.7 Greška pri brisanju (DbUpdateException)

Pitanje:

Zašto se javlja greška prilikom brisanja kursa?

Odgovor:

Greška nastaje zbog povezanih tabela:

- Lessons
- Exams
- Results
- Reviews

Rješenje:

- obrisati zavisne podatke prije brisanja ili
 - koristiti ON DELETE CASCADE
-

2.8 Foreign Key problem

Pitanje:

Kako riješiti problem foreign key constraint-a?

Odgovor:

Opcije:

1. Ručno brisanje povezanih podataka
2. Dodavanje cascade delete:

ON DELETE CASCADE

2.9 Dinamički navbar

Pitanje:

Kako prikazati različit navbar za admina i korisnika?

Odgovor:

Koristi se Session:

```
if ((int)Session["RoleID"] == 1)
{
    // admin meni
}
else
{
    // user meni
}
```

2.10 Notifikacije sistem

Pitanje:

Kako implementirati notifikacije?

Odgovor:

- SQL trigger ubacuje notifikacije
- MVC čita iz baze
- prikazuje broj nepročitanih u navbaru